

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №11.

Тема: «Последовательные контейнеры библиотеки STL»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

13	Задача 1 1. Контейнер - двунаправленная очередь 2. Тип элементов - double Задача 2 Тип элементов Pair (см. лабораторную работу №3). Задача 3 Параметризованный класс – Список (см. лабораторную работу №7) Задача 4 Адаптер контейнера – стек. Задача 5 Параметризованный класс – Список Адаптер контейнера – стек.		
	Задание 3	Задание 4	Задание 5
	Найти максимальный элемент и добавить его в конец контейнера	Найти элементы ключами из заданного диапазона и удалить их из контейнера	К каждому элементу добавить среднее арифметическое контейнера.

Анализ задачи:

Задача 1.

- Создаем последовательный контейнер.
- Заполняем его элементами стандартного типа.
- Добавляем элементы в соответствии с заданием
- Удаляем элементы в соответствии с заданием.

Задача 2.

- Создаем последовательный контейнер.
- Заполняем его элементами пользовательского типа (тип указан в варианте). Для пользовательского типа перегружаем необходимые операции.
- Добавляем элементы в соответствии с заданием
- Удаляем элементы в соответствии с заданием.

Задача 3

- Создаем параметризованный класс, используя в качестве контейнера последовательный контейнер.
- Заполняем его элементами.
- Добавляем элементы в соответствии с заданием
- Удаляем элементы в соответствии с заданием.

Задача 4

- Создаем адаптер контейнера.

- Заполняем его элементами пользовательского типа (тип указан в варианте). Для пользовательского типа перегружаем необходимые операции.
- Добавляем элементы в соответствии с заданием
- Удаляем элементы в соответствии с заданием.

Задача 5

- Создаем параметризованный класс, используя в качестве контейнера адаптер контейнера.
- Заполняем его элементами.
- Добавляем элементы в соответствии с заданием
- Удаляем элементы в соответствии с заданием.

Код:

```
#include<iostream>
#include<deque>
#include<stack>
#include<vector>
#include<cstdlib>
#include<ctime>
using namespace std;

// ===== КЛАСС PAIR =====

class Pair {
private:
    double first;
    double second;
public:
    Pair() { first = 0; second = 0; }
    Pair(double f, double s) { first = f; second = s; }
    Pair(const Pair& p) { first = p.first; second = p.second; }
    ~Pair() {}

    double getFirst() const { return first; }
    double getSecond() const { return second; }
    void setFirst(double f) { first = f; }
    void setSecond(double s) { second = s; }
```

// Перегруженные операции

```
Pair& operator=(const Pair& p) {  
    if (&p == this) return *this;  
    first = p.first;  
    second = p.second;  
    return *this;  
}
```

```
Pair operator+(const Pair& p) const {  
    return Pair(first + p.first, second + p.second);  
}
```

```
Pair operator/(const double& d) const {  
    return Pair(first / d, second / d);  
}
```

```
bool operator>(const Pair& p) const {  
    return (first > p.first) || (first == p.first && second > p.second);  
}
```

```
bool operator<(const Pair& p) const {  
    return (first < p.first) || (first == p.first && second < p.second);  
}
```

```
bool operator>=(const Pair& p) const { return !(*this < p); }
```

```
bool operator<=(const Pair& p) const { return !(*this > p); }
```

```
bool operator==(const Pair& p) const { return first == p.first && second == p.second; }
```

```
friend istream& operator>>(istream& in, Pair& p) {  
    cout << "first? "; in >> p.first;  
    cout << "second? "; in >> p.second;  
    return in;  
}
```

```
friend ostream& operator<<(ostream& out, const Pair& p) {
```

```

        out << "(" << p.first << ", " << p.second << ")";

        return out;
    }
};

// ===== ЗАДАЧА 1: deque<double> =====

typedef deque<double> Deque1;

Deque1 make_deque1(int n) {
    Deque1 d;
    for (int i = 0; i < n; i++) {
        double a = rand() % 100 - 50;
        d.push_back(a);
    }
    return d;
}

void print_deque1(const Deque1& d) {
    for (size_t i = 0; i < d.size(); i++)
        cout << d[i] << " ";
    cout << endl;
}

void task1_add_max(Deque1& d) {
    if (d.empty()) return;
    double max = d[0];
    for (size_t i = 1; i < d.size(); i++)
        if (d[i] > max) max = d[i];
    d.push_back(max);
    cout << "Максимальный элемент (" << max << ") добавлен в конец" << endl;
}

void task1_delete_range(Deque1& d, double key1, double key2) {
    if (key1 > key2) swap(key1, key2);
    int count = 0;
    for (Deque1::iterator it = d.begin(); it != d.end(); ) {

```

```

        if (*it >= key1 && *it <= key2) {
            it = d.erase(it);
            count++;
        }
        else {
            ++it;
        }
    }
    cout << "Удалено элементов: " << count << endl;
}

```

```

void task1_add_average(Deque1& d) {
    if (d.empty()) return;
    double sum = 0;
    for (size_t i = 0; i < d.size(); i++)
        sum += d[i];
    double avg = sum / d.size();
    for (size_t i = 0; i < d.size(); i++)
        d[i] += avg;
    cout << "Среднее арифметическое (" << avg << ") добавлено к каждому элементу" << endl;
}

```

```

void run_task1() {
    cout << "\n===== ЗАДАЧА 1: deque<double> =====" << endl;
    Deque1 d;
    int n;
    cout << "Введите количество элементов: ";
    cin >> n;
    d = make_deque1(n);
    cout << "Исходный deque: ";
    print_deque1(d);

    int choice;

    cout << "\nВыберите задание:\n";
    cout << "1. Задание 3 - Добавить максимальный элемент в конец\n";
    cout << "2. Задание 4 - Удалить элементы из диапазона\n";
}

```

```

cout << "3. Задание 5 - Добавить среднее арифметическое\n";
cout << "Ваш выбор: ";
cin >> choice;

switch (choice) {
case 1:
    task1_add_max(d);
    cout << "Результат: ";
    print_deque1(d);
    break;
case 2: {
    double key1, key2;
    cout << "Введите начало диапазона: ";
    cin >> key1;
    cout << "Введите конец диапазона: ";
    cin >> key2;
    task1_delete_range(d, key1, key2);
    cout << "Результат: ";
    print_deque1(d);
    break;
}
case 3:
    task1_add_average(d);
    cout << "Результат: ";
    print_deque1(d);
    break;
default:
    cout << "Неверный выбор!" << endl;
}
}

// ===== ЗАДАЧА 2: deque<Pair> =====
typedef deque<Pair> Deque2;

Deque2 make_deque2(int n) {
    Deque2 d;

```

```

    for (int i = 0; i < n; i++) {
        double f = rand() % 50;
        double s = rand() % 50;
        d.push_back(Pair(f, s));
    }
    return d;
}

```

```

void print_deque2(const Deque2& d) {
    for (size_t i = 0; i < d.size(); i++)
        cout << d[i] << " ";
    cout << endl;
}

```

```

void task2_add_max(Deque2& d) {
    if (d.empty()) return;
    Pair max = d[0];
    for (size_t i = 1; i < d.size(); i++)
        if (d[i] > max) max = d[i];
    d.push_back(max);
    cout << "Максимальный элемент (" << max << ") добавлен в конец" << endl;
}

```

```

void task2_delete_range(Deque2& d, Pair key1, Pair key2) {
    if (key1 > key2) swap(key1, key2);
    int count = 0;
    for (Deque2::iterator it = d.begin(); it != d.end(); ) {
        if (*it >= key1 && *it <= key2) {
            it = d.erase(it);
            count++;
        }
        else {
            ++it;
        }
    }
    cout << "Удалено элементов: " << count << endl;
}

```



```
}
```

```
void task2_add_average(Deque2& d) {  
    if (d.empty()) return;  
    Pair sum(0, 0);  
    for (size_t i = 0; i < d.size(); i++)  
        sum = sum + d[i];  
    Pair avg = sum / d.size();  
    for (size_t i = 0; i < d.size(); i++)  
        d[i] = d[i] + avg;  
    cout << "Среднее арифметическое добавлено к каждому элементу" << endl;  
}
```

```
void run_task2() {  
    cout << "\n===== ЗАДАЧА 2: deque<Pair> =====" << endl;  
    Deque2 d;  
    int n;  
    cout << "Введите количество элементов: ";  
    cin >> n;  
    d = make_deque2(n);  
    cout << "Исходный deque: ";  
    print_deque2(d);  
  
    int choice;  
    cout << "\nВыберите задание:\n";  
    cout << "1. Задание 3 - Добавить максимальный элемент в конец\n";  
    cout << "2. Задание 4 - Удалить элементы из диапазона\n";  
    cout << "3. Задание 5 - Добавить среднее арифметическое\n";  
    cout << "Ваш выбор: ";  
    cin >> choice;  
  
    switch (choice) {  
    case 1:  
        task2_add_max(d);  
        cout << "Результат: ";  
        print_deque2(d);  
    }
```

```

        break;
    case 2: {
        Pair key1, key2;

        cout << "Введите начало диапазона (Pair):\n";
        cin >> key1;

        cout << "Введите конец диапазона (Pair):\n";
        cin >> key2;

        task2_delete_range(d, key1, key2);

        cout << "Результат: ";
        print_deque2(d);

        break;
    }
    case 3:
        task2_add_average(d);
        cout << "Результат: ";
        print_deque2(d);
        break;
    default:
        cout << "Неверный выбор!" << endl;
    }
}

```

// ===== ЗАДАЧА 3: Параметризованный класс List =====

```

template<class T>
class List {
private:
    deque<T> lst;
public:
    List() {}
    List(int n) {
        T a;
        for (int i = 0; i < n; i++) {
            cin >> a;
            lst.push_back(a);
        }
    }
}

```

```

void Print() {
    for (typename deque<T>::iterator it = lst.begin(); it != lst.end(); ++it)
        cout << *it << " ";
    cout << endl;
}

```

```

void AddMaxToEnd() {
    if (lst.empty()) return;
    T max = *lst.begin();
    for (typename deque<T>::iterator it = lst.begin(); it != lst.end(); ++it)
        if (*it > max) max = *it;
    lst.push_back(max);
    cout << "Максимальный элемент добавлен в конец" << endl;
}

```

```

void DeleteRange(T key1, T key2) {
    if (key1 > key2) swap(key1, key2);
    int count = 0;
    for (typename deque<T>::iterator it = lst.begin(); it != lst.end(); ) {
        if (*it >= key1 && *it <= key2) {
            it = lst.erase(it);
            count++;
        }
        else {
            ++it;
        }
    }
    cout << "Удалено элементов: " << count << endl;
}

```

```

void AddAverage() {
    if (lst.empty()) return;
    T sum = *lst.begin();
    int count = 1;
    for (typename deque<T>::iterator it = ++lst.begin(); it != lst.end(); ++it) {

```

```

        sum = sum + *it;

        count++;

    }

    T avg = sum / count;

    for (typename deque<T>::iterator it = lst.begin(); it != lst.end(); ++it)

        *it = *it + avg;

    cout << "Среднее арифметическое добавлено к каждому элементу" << endl;

}

};

void run_task3() {

    cout << "\n===== ЗАДАЧА 3: Параметризованный класс List =====" << endl;

    int n;

    cout << "Введите количество элементов: ";

    cin >> n;

    List<double> lst(n);

    cout << "Исходный список: ";

    lst.Print();

    int choice;

    cout << "\nВыберите задание:\n";

    cout << "1. Задание 3 - Добавить максимальный элемент в конец\n";

    cout << "2. Задание 4 - Удалить элементы из диапазона\n";

    cout << "3. Задание 5 - Добавить среднее арифметическое\n";

    cout << "Ваш выбор: ";

    cin >> choice;

    switch (choice) {

    case 1:

        lst.AddMaxToEnd();

        cout << "Результат: ";

        lst.Print();

        break;

    case 2: {

        double key1, key2;

        cout << "Введите начало диапазона: ";

```

```

        cin >> key1;
        cout << "Введите конец диапазона: ";
        cin >> key2;
        lst.DeleteRange(key1, key2);
        cout << "Результат: ";
        lst.Print();
        break;
    }
    case 3:
        lst.AddAverage();
        cout << "Результат: ";
        lst.Print();
        break;
    default:
        cout << "Неверный выбор!" << endl;
    }
}

```

// ===== ЗАДАЧА 4: stack<Pair> =====

```

typedef stack<Pair> Stack;
typedef vector<Pair> Vector;

```

```

Vector copy_stack_to_vector(Stack s) {
    Vector v;
    while (!s.empty()) {
        v.push_back(s.top());
        s.pop();
    }
    return v;
}

```

```

Stack copy_vector_to_stack(Vector v) {
    Stack s;
    for (int i = v.size() - 1; i >= 0; i--)
        s.push(v[i]);
    return s;
}

```

```
}
```

```
Stack make_stack(int n) {  
    Stack s;  
    Pair p;  
    for (int i = 0; i < n; i++) {  
        cout << "Элемент " << i + 1 << ":\n";  
        cin >> p;  
        s.push(p);  
    }  
    return s;  
}
```

```
void print_stack(Stack s) {  
    Vector v = copy_stack_to_vector(s);  
    cout << "Стек (сверху вниз): ";  
    for (int i = v.size() - 1; i >= 0; i--)  
        cout << v[i] << " ";  
    cout << endl;  
}
```

```
void task4_add_max(Stack& s) {  
    if (s.empty()) return;  
    Vector v = copy_stack_to_vector(s);  
    Pair max = v[0];  
    for (size_t i = 1; i < v.size(); i++)  
        if (v[i] > max) max = v[i];  
    s = copy_vector_to_stack(v);  
    s.push(max);  
    cout << "Максимальный элемент добавлен в вершину стека" << endl;  
}
```

```
void task4_delete_range(Stack& s, Pair key1, Pair key2) {  
    if (key1 > key2) swap(key1, key2);  
    Vector v = copy_stack_to_vector(s);  
    Vector result;
```

```

for (int i = v.size() - 1; i >= 0; i--) {
    if (!(v[i] >= key1 && v[i] <= key2))
        result.push_back(v[i]);
}
s = copy_vector_to_stack(result);
cout << "Элементы из диапазона удалены" << endl;
}

```

```

void task4_add_average(Stack& s) {
    if (s.empty()) return;
    Vector v = copy_stack_to_vector(s);
    Pair sum(0, 0);
    for (size_t i = 0; i < v.size(); i++)
        sum = sum + v[i];
    Pair avg = sum / v.size();
    for (size_t i = 0; i < v.size(); i++)
        v[i] = v[i] + avg;
    s = copy_vector_to_stack(v);
    cout << "Среднее арифметическое добавлено к каждому элементу" << endl;
}

```

```

void run_task4() {
    cout << "\n===== ЗАДАЧА 4: stack<Pair> =====" << endl;
    Stack s;
    int n;
    cout << "Введите количество элементов: ";
    cin >> n;
    s = make_stack(n);
    cout << "Исходный стек: ";
    print_stack(s);

    int choice;
    cout << "\nВыберите задание:\n";
    cout << "1. Задание 3 - Добавить максимальный элемент в вершину\n";
    cout << "2. Задание 4 - Удалить элементы из диапазона\n";
    cout << "3. Задание 5 - Добавить среднее арифметическое\n";
}

```

```

cout << "Ваш выбор: ";
cin >> choice;

switch (choice) {
case 1:
    task4_add_max(s);
    cout << "Результат: ";
    print_stack(s);
    break;
case 2: {
    Pair key1, key2;
    cout << "Введите начало диапазона (Pair):\n";
    cin >> key1;
    cout << "Введите конец диапазона (Pair):\n";
    cin >> key2;
    task4_delete_range(s, key1, key2);
    cout << "Результат: ";
    print_stack(s);
    break;
}
case 3:
    task4_add_average(s);
    cout << "Результат: ";
    print_stack(s);
    break;
default:
    cout << "Неверный выбор!" << endl;
}
}

// ===== ЗАДАЧА 5: Параметризованный класс со stack =====

template<class T>
class ListWithStack {
private:
    stack<T> s;

```



```

vector<T> copy_stack_to_vector(stack<T> st) {
    vector<T> v;
    while (!st.empty()) {
        v.push_back(st.top());
        st.pop();
    }
    return v;
}

```

```

stack<T> copy_vector_to_stack(vector<T> v) {
    stack<T> st;
    for (int i = v.size() - 1; i >= 0; i--)
        st.push(v[i]);
    return st;
}

```

public:

```

ListWithStack() {}
ListWithStack(int n) {
    T a;
    for (int i = 0; i < n; i++) {
        cin >> a;
        s.push(a);
    }
}

```

```

void Print() {
    vector<T> v = copy_stack_to_vector(s);
    cout << "Стек (сверху вниз): ";
    for (int i = v.size() - 1; i >= 0; i--)
        cout << v[i] << " ";
    cout << endl;
    s = copy_vector_to_stack(v);
}

```

```

void AddMaxToEnd() {

```

```

    if (s.empty()) return;
    vector<T> v = copy_stack_to_vector(s);
    T max = v[0];
    for (size_t i = 1; i < v.size(); i++)
        if (v[i] > max) max = v[i];
    s = copy_vector_to_stack(v);
    s.push(max);
    cout << "Максимальный элемент добавлен в вершину" << endl;
}

```

```

void DeleteRange(T key1, T key2) {
    if (key1 > key2) swap(key1, key2);
    vector<T> v = copy_stack_to_vector(s);
    vector<T> result;
    for (int i = v.size() - 1; i >= 0; i--) {
        if (!(v[i] >= key1 && v[i] <= key2))
            result.push_back(v[i]);
    }
    s = copy_vector_to_stack(result);
    cout << "Элементы из диапазона удалены" << endl;
}

```

```

void AddAverage() {
    if (s.empty()) return;
    vector<T> v = copy_stack_to_vector(s);
    T sum = v[0];
    for (size_t i = 1; i < v.size(); i++)
        sum = sum + v[i];
    T avg = sum / v.size();
    for (size_t i = 0; i < v.size(); i++)
        v[i] = v[i] + avg;
    s = copy_vector_to_stack(v);
    cout << "Среднее арифметическое добавлено к каждому элементу" << endl;
}
};

```

```

void run_task5() {
    cout << "\n===== ЗАДАЧА 5: Параметризованный класс со stack =====" << endl;
    int n;
    cout << "Введите количество элементов: ";
    cin >> n;
    ListWithStack<double> lst(n);
    cout << "Исходный стек: ";
    lst.Print();

    int choice;
    cout << "\nВыберите задание:\n";
    cout << "1. Задание 3 - Добавить максимальный элемент в вершину\n";
    cout << "2. Задание 4 - Удалить элементы из диапазона\n";
    cout << "3. Задание 5 - Добавить среднее арифметическое\n";
    cout << "Ваш выбор: ";
    cin >> choice;

    switch (choice) {
    case 1:
        lst.AddMaxToEnd();
        cout << "Результат: ";
        lst.Print();
        break;
    case 2: {
        double key1, key2;
        cout << "Введите начало диапазона: ";
        cin >> key1;
        cout << "Введите конец диапазона: ";
        cin >> key2;
        lst.DeleteRange(key1, key2);
        cout << "Результат: ";
        lst.Print();
        break;
    }
    case 3:
        lst.AddAverage();

```

```

        cout << "Результат: ";

        lst.Print();

        break;

default:

        cout << "Неверный выбор!" << endl;

    }

}

// ===== ГЛАВНОЕ МЕНЮ =====

int main() {

    setlocale(LC_ALL, "ru");

    srand(time(0));

    int choice;

    do {

        cout << "\n===== \n";

        cout << "1. Задача 1 - deque<double>\n";

        cout << "2. Задача 2 - deque<Pair>\n";

        cout << "3. Задача 3 - Параметризованный класс List\n";

        cout << "4. Задача 4 - stack<Pair>\n";

        cout << "5. Задача 5 - Параметризованный класс со stack\n";

        cout << "0. Выход\n";

        cout << "===== \n";

        cout << "Выберите задачу: ";

        cin >> choice;

        switch (choice) {

            case 1:

                run_task1();

                break;

            case 2:

                run_task2();

                break;

            case 3:

                run_task3();

                break;

```

```
case 4:
    run_task4();
    break;
case 5:
    run_task5();
    break;
case 0:
    cout << "Выход из программы..." << endl;
    break;
default:
    cout << "Неверный выбор! Попробуйте снова." << endl;
}

if (choice != 0) {
    cout << "\nНажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
}

} while (choice != 0);

system("pause");
return 0;
}
```

Результаты работы:

```
=====
1. Задача 1 - deque<double>
2. Задача 2 - deque<Pair>
3. Задача 3 - Параметризованный класс List
4. Задача 4 - stack<Pair>
5. Задача 5 - Параметризованный класс со stack
0. Выход
=====
Выберите задачу: 1

===== ЗАДАЧА 1: deque<double> =====
Введите количество элементов: 8
Исходный deque: 42 -11 -27 -38 49 -39 6 22

Выберите задание:
1. Задание 3 - Добавить максимальный элемент в конец
2. Задание 4 - Удалить элементы из диапазона
3. Задание 5 - Добавить среднее арифметическое
Ваш выбор: 3
Среднее арифметическое (0.5) добавлено к каждому элементу
Результат: 42.5 -10.5 -26.5 -37.5 49.5 -38.5 6.5 22.5

Нажмите Enter для продолжения...

=====
1. Задача 1 - deque<double>
2. Задача 2 - deque<Pair>
3. Задача 3 - Параметризованный класс List
4. Задача 4 - stack<Pair>
5. Задача 5 - Параметризованный класс со stack
0. Выход
=====
Выберите задачу: 2

===== ЗАДАЧА 2: deque<Pair> =====
Введите количество элементов: 3
Исходный deque: (32, 11) (12, 25) (36, 37)

Выберите задание:
1. Задание 3 - Добавить максимальный элемент в конец
2. Задание 4 - Удалить элементы из диапазона
3. Задание 5 - Добавить среднее арифметическое
Ваш выбор: 1
Максимальный элемент ((36, 37)) добавлен в конец
Результат: (32, 11) (12, 25) (36, 37) (36, 37)

Нажмите Enter для продолжения...

=====
1. Задача 1 - deque<double>
2. Задача 2 - deque<Pair>
3. Задача 3 - Параметризованный класс List
4. Задача 4 - stack<Pair>
5. Задача 5 - Параметризованный класс со stack
0. Выход
=====
Выберите задачу: 3
```

```

===== ЗАДАЧА 3: Параметризованный класс List =====
Введите количество элементов: 4
2
3
4
5
Исходный список: 2 3 4 5

Выберите задание:
1. Задание 3 - Добавить максимальный элемент в конец
2. Задание 4 - Удалить элементы из диапазона
3. Задание 5 - Добавить среднее арифметическое
Ваш выбор: 2
Введите начало диапазона: 1
Введите конец диапазона: 2
Удалено элементов: 1
Результат: 3 4 5

Нажмите Enter для продолжения...

=====
1. Задача 1 - deque<double>
2. Задача 2 - deque<Pair>
3. Задача 3 - Параметризованный класс List
4. Задача 4 - stack<Pair>
5. Задача 5 - Параметризованный класс со stack
0. Выход
=====
Выберите задачу: 4

===== ЗАДАЧА 4: stack<Pair> =====
Введите количество элементов: 3
Элемент 1:
first? 4
second? 6
Элемент 2:
first? 7
second? 8
Элемент 3:
first? 6
second? 7
Исходный стек: Стек (сверху вниз): (4, 6) (7, 8) (6, 7)

Выберите задание:
1. Задание 3 - Добавить максимальный элемент в вершину
2. Задание 4 - Удалить элементы из диапазона
3. Задание 5 - Добавить среднее арифметическое
Ваш выбор: 3
Среднее арифметическое добавлено к каждому элементу
Результат: Стек (сверху вниз): (9.66667, 13) (12.6667, 15) (11.6667, 14)

Нажмите Enter для продолжения...

=====
1. Задача 1 - deque<double>
2. Задача 2 - deque<Pair>
3. Задача 3 - Параметризованный класс List
4. Задача 4 - stack<Pair>
5. Задача 5 - Параметризованный класс со stack
0. Выход
=====

```

Выберите задачу: 5

===== ЗАДАЧА 5: Параметризованный класс со stack =====

Введите количество элементов: 7

3

4

4

4

3

2

3

Исходный стек: Стек (сверху вниз): 3 4 4 4 3 2 3

Выберите задание:

1. Задание 3 - Добавить максимальный элемент в вершину

2. Задание 4 - Удалить элементы из диапазона

3. Задание 5 - Добавить среднее арифметическое

Ваш выбор: 3

Среднее арифметическое добавлено к каждому элементу

Результат: Стек (сверху вниз): 6.28571 7.28571 7.28571 7.28571 6.28571 5.28571 6.28571

Нажмите Enter для продолжения...

=====

1. Задача 1 - deque<double>

2. Задача 2 - deque<Pair>

3. Задача 3 - Параметризованный класс List

4. Задача 4 - stack<Pair>

5. Задача 5 - Параметризованный класс со stack

0. Выход

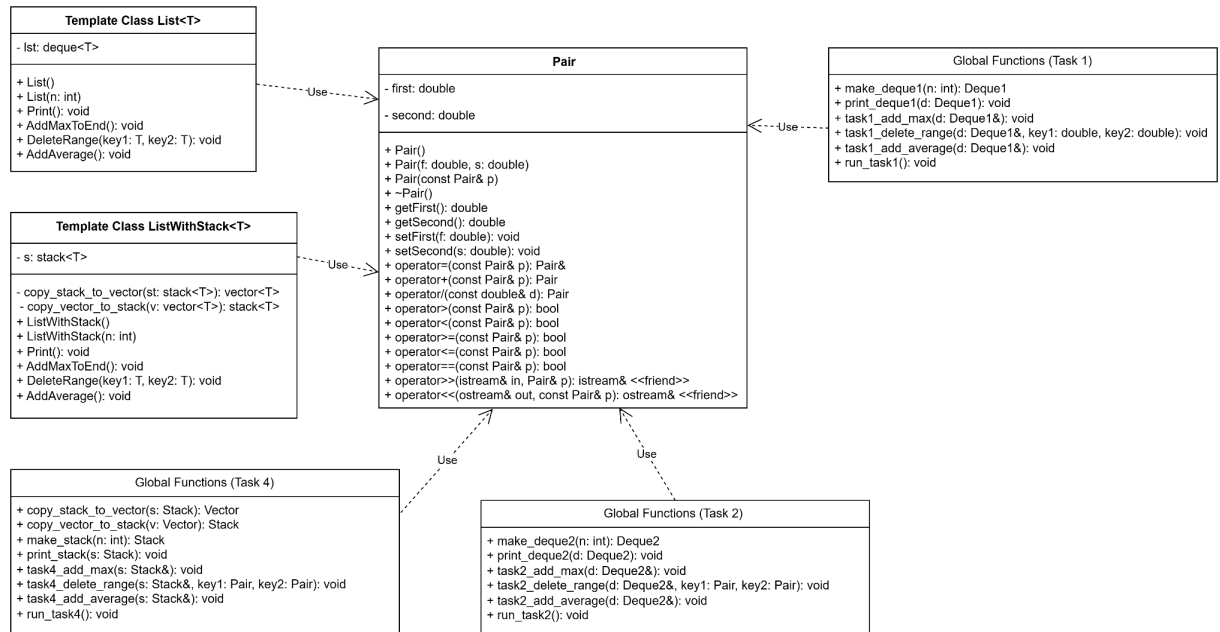
=====

Выберите задачу: 0

Выход из программы...

Для продолжения нажмите любую клавишу . . .

UML:



Вопросы:

1. STL состоит из двух частей: набор контейнерных классов и набор обобщенных алгоритмов.
2. Последовательные (vector, list, deque) и ассоциативные (map, multimap, set, multiset, bitset).
3. Подключить соответствующий заголовочный файл (#include <vector>, #include <list> и т.д.) и using namespace std.
4. Итератор – обобщение указателя, ссылается на элемент контейнера, позволяет перемещаться и разыменовываться.
5. Разыменование (*p), присваивание, сравнение (==, !=), префиксный/постфиксный инкремент (++p, p++).
6. for (iterator i = container.begin(); i != container.end(); ++i) { ... }
7. Входные, выходные, прямые, двунаправленные, произвольного доступа.
8. Операции ==, !=, =, clear, insert, erase, size, max_size, empty, begin, end, rbegin, rend.
9. Произвольный доступ по индексу [] и at, добавление/удаление в конце (push_back/pop_back) – эффективны. Вставка/удаление в середине – неэффективны (сдвиг элементов).
10. Вставка и удаление в любой позиции эффективны (O(1)). Произвольный доступ отсутствует.

11. Эффективны вставка и удаление как в начале, так и в конце (push_front/pop_front, push_back/pop_back). Доступ по индексу менее эффективен, чем у вектора.
12. push_back, pop_back, insert, erase, [], at, swap, clear.
13. push_back, pop_back, push_front, pop_front, insert, erase, splice, swap, clear.
14. push_back, pop_back, push_front, pop_front, insert, erase, [], at, swap, clear.
15. v.erase(v.begin()+2, v.begin()+5);
16. v.pop_back() или v.erase(v.end()-1).
17. list::iterator first = l.begin(); advance(first,2); list::iterator last = l.begin(); advance(last,5); l.erase(first, last);
18. l.pop_back() или l.erase(--l.end()).
19. d.erase(d.begin()+2, d.begin()+5);
20. d.pop_back() или d.erase(d.end()-1).
21. template<class T> void print(T &c) { for (auto p = c.begin(); p != c.end(); ++p) cout << *p << " "; }
22. Адаптеры – специализированные контейнеры, реализованные на основе базовых (stack, queue, priority_queue).
23. stack<int> s – стек на основе deque (по умолчанию); stack<int, list<int>> s – на основе списка.
24. push, pop, top, empty, size.
25. push, pop, front, back, empty, size.
26. queue – FIFO; priority_queue – при извлечении выбирается максимальный элемент (по приоритету).
27. Не поддерживает произвольный доступ. Нужно переложить элементы во вспомогательный контейнер, пропустить нужный, затем восстановить.
28. Аналогично – нет доступа по номеру; требуется переключивание во вспомогательный контейнер.
29. template<class T> void print_stack(stack<T> s) { while (!s.empty()) { cout << s.top() << " "; s.pop(); } }
30. template<class T> void print_queue(queue<T> q) { while (!q.empty()) { cout << q.front() << " "; q.pop(); } }